

Independent Exhaustive Verification of OEIS A369431 through $n = 22$ and a Formula-Derived b-File through $n = 400$

Carlo Corti
Independent researcher
`carlo.corti@outlook.com`

July 22, 2026

Abstract

OEIS sequence A369431 counts permutations of $[n]$ avoiding the four classical patterns 1234, 1324, 1342, and 2413. An independently implemented, formula-free maximum-insertion program exhaustively computes

$$a(21) = 8321204878, \quad a(22) = 26674199972.$$

Every avoiding permutation is reached exactly once by repeatedly inserting the next maximum into an avoiding parent. Because the parent already avoids the forbidden set, the incremental predicate need inspect only new occurrences that contain the inserted maximum. Exhaustive coverage is organized through the 20 order-4 avoiders and their 66 order-5 children, while a small fixed number of operational groups provides restartable human-scale runs without fragmenting the mathematical computation or its archival record. The two reported totals are supported by append-only manifests, final aggregate tables, small-order direct oracles and replay tests, sanitizer and static analysis, and external LLM-assisted code reviews. A subsequent literature search found that PermPAL had already recorded a verified combinatorial specification and rational generating function for this exact class. We use that prior result, with explicit attribution, to produce an OEIS-style b-file through $n = 400$. Exact Python recurrence iteration and an independent PARI/GP formal-series expansion agree for all 401 coefficients; the first 101 also agree with the sequence displayed by PermPAL. No priority is claimed for the formula, recurrence, or coefficients. The contribution is instead a transparent separation of a finite independent enumeration through $n = 22$ from a reproducible formula-derived data extension thereafter.

2020 Mathematics Subject Classification. Primary 05A05; Secondary 05A15, 11Y55.

Key words and phrases. permutation patterns, pattern avoidance, exact enumeration, rational generating function, integer sequence, OEIS, reproducible computation.

1 Introduction

We write $[n] = \{1, \dots, n\}$ and call n the order (or length) of a permutation. Let

$$B = \{1234, 1324, 1342, 2413\}$$

and let $\text{Av}_n(B)$ denote the permutations of $[n]$ that avoid every classical pattern in B . Since the four distinct forbidden patterns have the same length, they form an antichain, and each is necessary;

hence B is also the basis of this permutation class. The OEIS entry A369431 is

$$a(n) = |\text{Av}_n(B)|. \tag{1}$$

The local OEIS snapshot has offset 0, with $a(0) = 1$ for the empty permutation, and gives the terms through

$$a(20) = 2595858574.$$

It credits Matt Slattery-Holmes with submitting the sequence on January 23, 2024, and Martin Ehrenstein with extending it from $a(13)$ through $a(20)$ on February 24, 2024 [9]. For example, at $n = 4$ precisely the four members of B are excluded from the 24 permutations, so $a(4) = 20$.

Classical pattern avoidance belongs to the established theory of restricted permutations, with foundational results due to Simion and Schmidt and modern book-length treatments by Bóna and Kitaev [14, 3, 6]. MathWorld provides a concise online introduction and links pattern-avoidance counts to their OEIS records [15]. These sources supply the general context.

A literature and database search prompted by the first review round located this exact class in the Permutation Pattern Avoidance Library (PermPAL), where it is accompanied by an automatically verified combinatorial specification, a rational generating function, and a counting sequence that includes the two values computed here [2, 10]. Thus the novelty claimed here is not priority for those integers, the recurrence, or the generating function. The contribution is the independent, formula-free exhaustive check through $n = 22$, its compact audit trail, and a separately labelled, reproducible b-file calculation from the prior structural result.

The principal difficulty in a direct extension is the number of avoiding objects themselves. Enumerating all $n!$ permutations is unnecessary, but a formula-free generator must still reach every avoiding permutation and must do so without omission or duplication. The computation described here uses the hereditary nature of classical pattern avoidance: it starts from a small avoiding permutation and inserts successive maxima, pruning an insertion as soon as it creates one of the four forbidden patterns.

Successive insertion is also central to the insertion-encoding approach to permutation classes developed by Albert, Linton, and Ruškuc [1]. The present method is deliberately narrower: it does not construct a regular-language encoding, infer a generating function, or consume the PermPAL specification. It uses a fixed maximum-insertion tree and a basis-specific direct predicate solely for exact finite enumeration.

The contributions of this paper are:

1. a direct correctness argument for a sequence-specific incremental maximum-insertion predicate;
2. a complete and disjoint coverage scheme based on 20 mathematical atoms of order 4, refined into 66 durable internal units of order 5;
3. an operational grouping and persistence design that retains one monolithic production source, one manifest per target, and one final aggregate per target;
4. an artifact-bounded, independently implemented computation of the two values stated in Theorem 3; and
5. a formula-derived b-file through $n = 400$, cross-checked by two exact calculation paths and against all 101 coefficients displayed by PermPAL.

The exhaustive C computation does not depend on a recurrence or generating function. The later b-file calculation in Section 7 does; the two evidential tracks are kept explicit throughout.

2 Pattern avoidance and maximum insertion

Definition 1. Let $\pi = \pi_1 \cdots \pi_n$ be a permutation and let $\sigma = \sigma_1 \cdots \sigma_k$ be a pattern. The permutation π *contains* σ if some subsequence $\pi_{i_1}, \dots, \pi_{i_k}$, with $i_1 < \cdots < i_k$, has the same relative order as σ . Otherwise π *avoids* σ .

Suppose that $\pi \in \text{Av}_m(B)$ and form q by inserting the new maximum $M = m + 1$ at position p . An occurrence of a forbidden pattern that does not use M would already occur in π . Thus a new forbidden occurrence must contain M . Since M is the largest of the four values, it corresponds to the entry 4 in each forbidden pattern. The four possible cases are exactly

$$1234 : i < j < k < p, \quad q_i < q_j < q_k < M, \quad (2)$$

$$1324 : i < j < k < p, \quad q_i < q_k < q_j < M, \quad (3)$$

$$1342 : i < j < p < k, \quad q_i < q_k < q_j < M, \quad (4)$$

$$2413 : i < p < j < k, \quad q_j < q_i < q_k < M. \quad (5)$$

The production predicate is the negation of the existence of any index tuple satisfying one of (2)–(5). It is therefore a fixed predicate for this basis, not a generic pattern parser.

Proposition 2 (Correctness of maximum insertion). *For every $n \geq 1$, recursively inserting the next maximum in every position that passes the four tests above generates every member of $\text{Av}_n(B)$ exactly once and generates no other permutation.*

Proof. Proceed by induction on the order. The initial permutation of order 1 is the unique member of $\text{Av}_1(B)$ and avoids B . If an avoiding parent is extended, an old forbidden occurrence cannot appear; equations (2)–(5) enumerate all possible new forbidden occurrences containing the inserted maximum. Consequently, the predicate accepts exactly the avoiding children.

Conversely, delete the maximum from any $q \in \text{Av}_{m+1}(B)$. The result is an avoiding permutation π of order m , because classical pattern avoidance is hereditary. Reinserting $m + 1$ at its original position recovers q and passes the incremental test. Deletion of the maximum is a unique inverse because the maximum has one uniquely determined position. Thus the construction is both complete and duplicate-free. Heredity also ensures that every intermediate permutation on an insertion path is an avoider, so the incremental premise is valid at every level. The recursion terminates because the order increases by one at every step and stops at the fixed target n . $\square$

This proposition establishes the mathematical enumeration scheme. The finite numerical conclusions additionally depend on the faithful C17 implementation and the recorded completed runs, whose validation boundary is described below.

3 Coverage atoms, internal units, and operational groups

3.1 Twenty mathematical coverage atoms

There are $a(4) = 20$ avoiders of order 4. They form a natural disjoint frontier: repeatedly deleting the current maximum from any avoiding permutation of order at least 4 ends at exactly one member of $\text{Av}_4(B)$. The program stores these 20 roots explicitly in lexicographic order. They are *mathematical coverage atoms*; they are not 20 source files, 20 manifests, or a requirement for 20 operator launches.

Indeed, Proposition 2 shows that deleting the maximum maps an avoider to its unique avoiding parent. Iterating this map down to order 4 assigns every target-order avoider to exactly one of the

20 roots. Consequently the 20 rooted subtrees are pairwise disjoint and their union is $\text{Av}_n(B)$ for every $n \geq 4$.

3.2 The order-5 recovery frontier

For targets above 4, each valid insertion of 5 into the 20 roots defines one internal unit. Roots are numbered $0, \dots, 19$ in lexicographic order; the five insertion positions are numbered $0, \dots, 4$, from before the first entry through after the last. Ordered first by root and then by insertion position, the accepted children are numbered $u = 0, \dots, 65$ and are exactly $\text{Av}_5(B)$; hence they number

$$a(5) = 66.$$

By Proposition 2, their descendant subtrees are complete, pairwise disjoint, and collectively cover the target order. The refinement from 20 atoms to 66 units is useful for recovery: after an interruption, a completed unit need not be counted again. It does not change the underlying mathematical partition.

3.3 Segmentation without fragmentation

The 66 units are bundled into a small fixed number G of operational groups, numbered $0, \dots, G-1$: $G = 4$ for the $n = 21$ campaign and $G = 8$ for the $n = 22$ campaign. Unit u belongs to group $u \bmod G$, which interleaves neighboring lexicographic units and reduces the imbalance that would result from assigning contiguous subtrees. One command processes one entire operational group and then exits. Thus the human stop/go boundary is a completed group, whereas the finer units are durable recovery boundaries inside that group.

The two values of G were fixed separately for the reported targets from measured unit throughput and the intended human-scale run duration. They are campaign parameters for these two completed computations, not a proposed general rule for targets that were never run.

The grouping is an operational partition only. It neither changes the incremental predicate nor filters the objects being counted. The $n = 21$ groups had recorded durations totaling 8757.72 seconds. For $n = 22$, the eight groups totaled 31789.90 seconds; seven were within the intended 30–90 minute range, and one took 5572.62 seconds, slightly above 90 minutes. These times are target-run telemetry reported in the post-result validation records, not a complexity claim or a forecast for an unexecuted target [4].

4 Implementation and persistent evidence

The production program is a single C source file compiled in C17 mode, `src/a369431.c`, built by `src/Makefile`. It is serial and contains the fixed basis, the 20 order-4 roots, the order-5 frontier construction, and the recursive counter. Permutation entries use fixed-size `uint8_t` arrays, while leaf and node counts use the GCC/Clang extension `unsigned __int128` with checked addition and explicit decimal conversion. Any failed checked addition terminates the unit with a nonzero outcome and records `counter-overflow`; silent wraparound is not accepted. The source-level campaign cap is $n \leq 24$, well inside both the fixed array capacity and the `uint8_t` value range. No recurrence generator, runtime pattern parser, external executable, subprocess, or general scheduler participates in the count.

At each recursive node, the program inserts the next maximum in all positions, applies the predicate of Section 2, and recurses only on accepted children. Each call on an accepted permutation, including the order-5 unit root and a target-order permutation, contributes one node; rejected

candidate insertions contribute none. A target-order node also contributes one leaf. Thus, for the reported targets, $\text{nodes}(n) = \sum_{m=5}^n a(m)$. The node counter supports telemetry and gives an additional aggregate invariant but does not affect acceptance.

Each target has one append-only manifest. Before counting a unit, the program appends a **running** row; after successful completion it appends a **complete** row containing leaf and node totals, elapsed time, coverage label, source and binary identities, timestamp, and stop reason. The manifest is locked during an invocation, checked on every restart, and refuses rows with foreign provenance or an invalid state transition. A completed unit is never silently overwritten. When and only when all 66 units are complete, the program sums their counts and publishes one no-clobber aggregate TSV for the target. The aggregate records complete group, unit, and atom coverage as well as the manifest identity and method.

The manifest schema is the 12-column TSV sequence **sequence**, **target**, **segment**, **coverage**, **status**, **leaves**, **nodes**, **elapsed_s**, **source_sha256**, **binary_sha256**, **epoch**, and **stop_reason**. For an order-5 unit, the **coverage** value is the explicit string **root=r,insert5=p**, identifying its order-4 parent and zero-based insertion position. The aggregate schema records sequence, target, coverage, leaf and node totals, source, binary, and manifest SHA-256 values, method, and recurrence status. These schemas are data contracts for the single manifest and aggregate, not separate per-unit files.

Long invocations print an immediate unit-start line, periodic heartbeats, and a unit-closure line with counts and elapsed time. This terminal telemetry does not create auxiliary log trees and is not part of the manifest/aggregate truth boundary. The durable public evidence therefore remains compact: one monolithic source, one Makefile, one manifest and one aggregate per target, the consolidated term tables, and the validation reports.

The bounded local verification commands are

```
make -C src clean test
make -C src sanitize
make -C src clang
make -C src static
```

from the public package root. The archived production interface processes a single group with **src/a369431 run TARGET GROUP**, or the next incomplete group with **src/a369431 run TARGET next**. Reproduction of a complete large campaign is intentionally distinct from the short validation commands: starting from an empty target manifest, four successful invocations of **src/a369431 run 21 next** close $n = 21$, and eight successful invocations of **src/a369431 run 22 next** close $n = 22$; the final invocation publishes the aggregate automatically. Reissuing **next** after an interruption reruns only the incomplete logical unit within the selected group and preserves completed units.

The target workstation ran Linux Mint 22.3 on an AMD Ryzen 9 7940HS (8 cores/16 threads) with 64 GB of installed DDR5 RAM. The reviewed Makefile default was **-O3 -std=c17** plus the warning flags printed in **src/Makefile**. Exact production kernel, compiler version, and any operator-supplied flag override were not retained; therefore the elapsed times in Section 3.3 are provenance telemetry, not portable performance benchmarks. This missing metadata is a reproducibility limitation rather than a value-correction claim.

5 Validation and certification boundary

Validation addressed the mathematical predicate, implementation, persistence, and final arithmetic through several partially independent checks. The main checks and their recorded scopes are summarized in Table 1.

For each target manifest, the post-result audit found 66 **running** and 66 **complete** rows, no missing or duplicate complete unit, and exact agreement between the sum of completed rows and the final aggregate. The source and binary identities in every row matched those of the reviewed canonical production candidate. At campaign closure, the then-current b-file and consolidated result table agreed term by term through $n = 22$. The public b-file has subsequently been extended by the separate formula-derived process in Section 7; the certified-term and exhaustive-result TSVs deliberately remain bounded at $n = 22$ [4].

The small direct predicate is mathematically different from the incremental test but is included in the same production source for selftest purposes. An external Claude review additionally reported an LLM-generated Python brute force comparison through $n = 9$, independent of the C implementation but not an independent human-authored verification. Neither check is a second exhaustive enumeration at $n = 21$ or $n = 22$. Likewise, the LLM reviews are code and artifact reviews, not independent mathematical peer review and not a second production campaign.

Accordingly, “validated” or “certified” in this paper has an explicit artifact-bounded meaning: Proposition 2 supplies the mathematical coverage argument; the reviewed implementation supplies the enumerator; and the manifests and aggregates certify completion and arithmetic consistency for that implementation. The residual common-mode risk is an implementation error not detected by the direct small-order oracle, known-term replay, compiler/sanitizer/static checks, or external code reviews.

The sanitizer stage included AddressSanitizer, the compiler-instrumentation method of Serebryany, Bruening, Potapenko, and Vyukov, together with UndefinedBehaviorSanitizer [13]. These tools test important memory-safety and undefined-behavior boundaries, but their successful execution is not by itself a proof of algorithmic correctness.

The theorem-critical source, binary, manifest, and aggregate SHA-256 identities are recorded in the package’s validation ledger and post-result reports [4]. Keeping the inventory in those machine-audit documents avoids turning the scientific narrative into a hash catalogue. Byte identities complement, but do not replace, a versioned public release and archival DOI.

Table 1: Validation evidence recorded for the canonical production source.

Check	Recorded scope and result
Root and frontier construction	The explicit 20 roots match the direct order-4 oracle; the valid order-5 frontier has 66 units.
Incremental predicate comparison	Exhaustive equality with the direct four-tuple predicate through order 9.
Direct small-order oracle	Known values reproduced through order 9.
Maximum-insertion replay	Known values reproduced through order 12; external review also recorded grouped known-term checks at orders 12 and 16.
Node-count invariant	Summing the consolidated terms according to $\text{nodes}(n) = \sum_{m=5}^n a(m)$ reproduced both aggregate node totals.
Persistence selftests	Manifest parsing, state transitions, interrupted groups, no-clobber aggregate behavior, group coverage, and numeric parsing passed bounded fixtures.
Compiler and analysis checks	GCC and Clang builds/selftests, ASan/UBSan, <code>clang-tidy</code> , and <code>cppcheck</code> passed under the recorded configurations [8, 7, 5].
Campaign closure	For each of $n = 21$ and $n = 22$, all 66 units were complete exactly once; manifest sums equaled aggregate leaf and node totals; coverage was 20/20 atoms.
External LLM-assisted reviews	ChatGPT and Claude reports examined the frozen grouping/publication candidate and recorded no reproducible defect in their stated scopes; this is review history, not independent peer review or numerical evidence.

6 Computational results

The final aggregate records are shown in Table 2. Here “nodes” is the inclusive accepted-node count defined in Section 4; it is a diagnostic total distinct from the number of target-order leaves.

Theorem 3 (Artifact-bounded computational result). *Subject to the correct execution of the reviewed production implementation described in Section 4 and the completed canonical manifests described in Section 5, exhaustive maximum-insertion enumeration gives*

$$|\text{Av}_{21}(B)| = 8321204878$$

and

$$|\text{Av}_{22}(B)| = 26674199972.$$

Computational proof. Proposition 2 proves that the maximum-insertion tree accepts exactly the avoiders and that every target permutation has a unique path. The 20 order-4 atoms cover all

Table 2: Canonical aggregate results.

n	Groups	Units	Leaves $a(n)$	Nodes
21	4	66	8321204878	12094019604
22	8	66	26674199972	38768219576

possible paths, and their 66 order-5 children give a disjoint refinement. For each target, the canonical manifest contains one completed count for every one of these 66 units. Post-result validation found no missing or duplicated complete unit and verified that the sums of the unit leaf and node counts equal the corresponding aggregate row. The aggregate leaf totals are those displayed in Table 2, which establishes the two finite computational claims within the certification boundary of Section 5. $\square$

The theorem gives a computation independent of the structural result below. The package’s `data/certified_terms.tsv` and `results/a369431_terms.tsv` stop at $n = 22$ so that the exhaustive certification boundary remains visible.

7 External structural result and formula-derived b-file

The PermPAL record for this exact class reports the rational generating function

$$A(x) = \sum_{n \geq 0} a(n)x^n = \frac{(x-1)(x^2-3x+1)}{4x^3-7x^2+5x-1}, \quad (6)$$

together with a machine-generated proof tree and counting sequence [2, 10]. The record states that its *Point Placements* specification has 51 rules and was found by the PermPAL system on January 18, 2022. This source was located only after the $n = 21$ and $n = 22$ campaigns and the initial manuscript had been completed. It was therefore absent from the production design and canonical execution path. The paper does not claim to have discovered or reproved (6).

Multiplying numerator and denominator by -1 rewrites (6) as

$$(1-5x+7x^2-4x^3)A(x) = 1-4x+4x^2-x^3.$$

Coefficient comparison gives $a(0), a(1), a(2), a(3) = 1, 1, 2, 6$ and, for $n \geq 4$,

$$a(n) = 5a(n-1) - 7a(n-2) + 4a(n-3). \quad (7)$$

At the two independently enumerated endpoints, the formula gives the post-hoc equalities

$$\begin{aligned} 5(2595858574) - 7(809796400) + 4(252621702) &= 8321204878, \\ 5(8321204878) - 7(2595858574) + 4(809796400) &= 26674199972. \end{aligned}$$

This agreement is not a generator, pruning rule, acceptance test, independent campaign certificate, or proof of Theorem 3.

For the separate data-extension task, however, the prior PermPAL formula is the declared generator. The Python 3.12.3 tool `tools/generate_b369431.py` iterates (7) with exact integers through $n = 400$, starting from the four displayed values $a(0), \dots, a(3) = 1, 1, 2, 6$. Independently, PARI/GP 2.15.4 expands (6) directly as a formal power series; its script does not iterate the Python

implementation [12, 11]. The two outputs agree byte for byte for all 401 coefficients. They also agree with the 101 coefficients displayed by PermPAL for $0 \leq n \leq 100$ and with the previous local b-file for $0 \leq n \leq 22$.

Thus the recurrence reproduces every term in the pre-campaign OEIS range $0 \leq n \leq 20$ as well as the two C endpoints. In this paper, “formula-derived” means a coefficient of the cited rational series, calculated and cross-checked locally; its validity remains conditional on that external generating function and does not carry the exhaustive C certificate.

The cutoff at $n = 400$ is editorial, not mathematical. Since the denominator of (6) has nonzero constant term, the rational function has a unique formal power-series expansion at the origin, and (7) determines every later coefficient from three preceding ones. Thus, conditional on the correctness of the cited PermPAL generating function, the calculation can be continued to any prescribed finite index; only integer size, running time, memory, and output size impose practical limits. We stop at 400 to keep the public data proportionate to the paper.

Representative new b-file rows are

n	$a(n)$
23	85506000010
24	274095419758
25	878631898608
40	34070992412092462900
50	3903342517082139278233270

The b-file continues through $a(400)$, a 202-digit integer. Its full value belongs in the machine-readable data rather than the running text. The validation report records the commands, comparisons, file format checks, and SHA-256 identities. This is a reproducible execution and verification of a previously published formula, not a claim of priority for the resulting coefficients.

8 Limitations

The reported certification is finite and tied to the archived source, manifests, aggregate tables, and validation records. There is no second independent exhaustive implementation reproducing the complete $n = 21$ and $n = 22$ searches. The direct oracle and LLM-generated Python check cover only small orders, while the known-term and grouped replays cover bounded larger orders. External LLM-assisted reviews strengthen the source and artifact audit but do not replace conventional mathematical peer review.

The exact production kernel, compiler version, and effective operator build flags were not retained, so the reported elapsed times cannot be reproduced as benchmarks even though source, binary identity, data, and completion records are pinned.

The exhaustive campaign ends at $a(22)$; no maximum-insertion campaign for $a(23)$ or any higher term was executed. Rows 23..400 of the b-file have the weaker and different status “derived from the PermPAL generating function.” Agreement between Python and PARI/GP protects against elementary transcription or implementation errors but is not an independent proof of the underlying 51-rule PermPAL specification cited in Section 7. Possible future work includes a human-readable derivation or independent audit of that specification and an independent exhaustive reproduction of the two finite counts; neither has been performed here.

9 Conclusion

Formula-free maximum insertion independently verifies

$$a(21) = 8321204878, \quad a(22) = 26674199972.$$

The key computational structure is the separation between mathematical coverage and operational recovery: 20 order-4 atoms prove coverage, 66 order-5 units provide durable disjoint subtasks, and only a few balanced groups define the operator-facing runs. This yields a compact auditable record without changing the enumeration. The previously published PermPAL generating function then permits a negligible-cost exact calculation of a b-file through $n = 400$. Two arbitrary-precision routes and the 101 displayed PermPAL coefficients cross-check that calculation. The scientific roles are therefore deliberately distinct: independent exhaustive evidence at $n = 21, 22$ and attributed formula-derived data at $n = 23, \dots, 400$.

Data availability

The public package contains the C code, b-file candidate, formula-generation scripts, certified-term table, result tables, append-only manifests, aggregate records, validation reports, and the manuscript as both `paper/A369431.tex` and `paper/A369431.pdf`. It is available from the GitHub repository <https://github.com/carcorti/A369431> and the Zenodo archive <https://doi.org/10.5281/zenodo.21485629>. The formula-derived extension is reproduced by `tools/generate_b369431.py` and `tools/generate_b369431.gp` and audited in `validation/A369431_formula_derived_bfile_validation.md`. The exhaustive evidence includes `data/b369431.txt`, the two manifests `validation/a369431_n21_manifest.tsv` and `validation/a369431_n22_manifest.tsv`, and the two aggregate records `results/a369431_n21.aggregate.tsv` and `results/a369431_n22.aggregate.tsv`. No per-unit directory or external dataset is needed to audit the reported finite computation.

AI use disclosure

Large language models were used as research, review, and drafting assistants during the A369431 workflow: OpenAI Codex and ChatGPT, Claude, DeepSeek, Gemini, Grok, Kimi, Qwen, MiniMax, and Z. Round-01 manuscript reports also included Lumo and Meta model families. The dated reports are retained in the local prepublication review archive. Names are given at family or tool level because exact model-version metadata were not consistently retained in the earlier local artifacts; the reports preserve the identifiers declared by each reviewer.

Mathematical analysis. DeepSeek contributed preliminary triage, and Codex assisted with reconciling the sequence definition, the maximum-insertion argument, and the distinction between the initially empirical recurrence and the later-located PermPAL result. Mathematical claims retained in this paper were checked against the local OEIS snapshot, the explicit four-case incremental predicate, the b-file, and the aggregate result tables. The recurrence was deliberately excluded from the exhaustive C generation and certification. It was subsequently used, with attribution, only for the formula-derived b-file described in Section 7.

Algorithm and code. Codex assisted with computational-strategy analysis, validation planning, code and artifact review, and integration of review evidence. ChatGPT and Claude performed external LLM-assisted reviews of the frozen production candidate; Claude also reported an independent Python small-order brute-force check, independent of the C implementation but not

human-authored. Acceptance of the final implementation was tied to the canonical C17 source and Makefile, selftests, direct-oracle and known-term comparisons, manifest/aggregate equality, compiler and sanitizer runs, static analysis, and the post-result validation reports. No conversational LLM assertion is used as numerical evidence.

Manuscript. Codex assisted with drafting, internal adversarial review, comparison of the 11 first-round reports, targeted literature research, successive manuscript revisions, the author-directed formula-extension revision, the formula-generation and validation workflow, and comparison of the 11 subsequent review reports. The reviewers received overlapping manuscript-centered packets and did not constitute independent human peer review. Every headline value and coverage statement was checked against the b-file candidate, certified-term and result TSVs, canonical manifests, aggregate TSVs, validation summaries, and code-review integration record. Formula-derived rows were additionally checked by exact Python and PARI/GP calculations and against PermPAL’s 101 displayed coefficients. The numerical claims rely on those artifacts and checks, not on generated prose.

The author takes full scientific responsibility for the mathematical statements, algorithm, implementation, numerical results, validation boundaries, and conclusions of this paper.

References

- [1] M. H. Albert, S. Linton, and N. Ruškuc, *The insertion encoding of permutations*, Electronic Journal of Combinatorics **12** (2005), Article R47. <https://doi.org/10.37236/1944>.
- [2] M. H. Albert, C. Bean, A. Claesson, É. Nadeau, J. Pantone, and H. Ulfarsson, *Combinatorial Exploration: An Algorithmic Framework for Enumeration*, Memoirs of the American Mathematical Society **317** (2026), no. 1611. ISBN 978-1-4704-8000-4.
- [3] M. Bóna, *Combinatorics of Permutations*, 3rd ed., Discrete Mathematics and Its Applications, CRC Press, 2022. ISBN 978-0-367-22258-1. <https://www.routledge.com/Combinatorics-of-Permutations/Bona/p/book/9780367222581>.
- [4] C. Corti, *OEIS A369431: Independent Maximum-Insertion Enumeration through $n = 22$ and Formula-Derived Data through $n = 400$* , GitHub and Zenodo release package, 2026. <https://github.com/carcorti/A369431>. <https://doi.org/10.5281/zenodo.21485629>.
- [5] Cppcheck project, *Cppcheck manual*, official documentation. <https://cppcheck.sourceforge.io/manual.html>. Accessed July 21, 2026.
- [6] S. Kitaev, *Patterns in Permutations and Words*, Monographs in Theoretical Computer Science, Springer, Berlin and Heidelberg, 2011. <https://doi.org/10.1007/978-3-642-17333-2>.
- [7] LLVM Project, *clang-tidy—Clang extra tools documentation*. <https://clang.llvm.org/extra/clang-tidy/>. Accessed July 21, 2026.
- [8] LLVM Project, *UndefinedBehaviorSanitizer documentation*. <https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html>. Accessed July 21, 2026.
- [9] OEIS Foundation Inc., *Sequence A369431: Number of permutations of $[n]$ avoiding 1234, 1324, 1342, and 2413*, The On-Line Encyclopedia of Integer Sequences, <https://oeis.org/A369431>. Submitted by Matt Slattery-Holmes, January 23, 2024; terms $a(13)$ – $a(20)$ contributed by Martin Ehrenstein, February 24, 2024. Local project snapshot consulted July 21, 2026.
- [10] Permutation Pattern Avoidance Library (PermPAL), *Av(1234, 1324, 1342, 2413)*, record 14688. https://www.permpal.com/perms/basis/0123_0213_0231_1302/. The page reports a 51-rule *Point Placements* specification found by the PermPAL system on January 18, 2022; accessed July 21, 2026.
- [11] The PARI Group, *PARI/GP User’s Guide, version 2.15.4*, Université de Bordeaux. <https://pari.math.u-bordeaux.fr/pub/pari/manuals/2.15.4/users.pdf>.
- [12] Python Software Foundation, *Python 3.12 documentation*. <https://docs.python.org/3.12/>. Accessed July 21, 2026.
- [13] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, *AddressSanitizer: A fast address sanity checker*, in *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, USENIX Association, 2012, pp. 309–318. <https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany>.

- [14] R. Simion and F. W. Schmidt, *Restricted permutations*, European Journal of Combinatorics **6** (1985), no. 4, 383–406. [https://doi.org/10.1016/S0195-6698\(85\)80052-4](https://doi.org/10.1016/S0195-6698(85)80052-4).
- [15] E. W. Weisstein, *Permutation Pattern*, MathWorld—A Wolfram Resource.
<https://mathworld.wolfram.com/PermutationPattern.html>. Accessed July 21, 2026.